Storage, caching, and scale

Week 6, Lecture 2 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

What we will cover

- SQL vs. NoSQL: how the access pattern determines the choice
- Three caching layers: client, CDN, and application cache
- Sharding patterns: range-based vs. hash-based and the hotspot tradeoff
- Queues and async work: when synchronous calls are the wrong tool
- CAP theorem: stating consistency tradeoffs without getting philosophical

Part 1: SQL vs. NoSQL

The question is always the access pattern

- SQL (relational): strong consistency, ACID transactions, flexible ad-hoc queries, joins.
- NoSQL (key-value, document, column): optimized for one access pattern at high throughput.
- ByteByteGo 101 (2023): “Which one should I use? It depends. Here is what it depends on.”
- The interview answer: name the access pattern first, then choose.

When SQL wins

- The workload needs joins across multiple entity types.
- The system requires ACID transactions (financial ledger, inventory, booking).
- The data model is not yet stable: schema changes are frequent.
- The scale is below 10 million rows and a single primary with read replicas handles it.

When NoSQL wins

- One dominant access pattern: look up by a single key, very fast, very often.
- Scale requires horizontal sharding across many machines (key-value scales out naturally).
- Write throughput is high and eventual consistency is acceptable (user activity logs, session store).
- The data is document-shaped and joins would be expensive or meaningless.

Comparing a URL shortener's choices

SQL (PostgreSQL)

- Schema: `(short_code, long_url, created_at, user_id)`
- Primary key lookup by `short_code` is fast
- Supports analytics joins: who created what, when
- One shard handles 182 GB easily in year one

Key-value (DynamoDB)

- Key: `short_code`, Value: `long_url`
- Lookup is O(1), extremely low latency at scale

Part 2: three caching layers

Layer 1: client cache

- Lives in the browser or mobile app.
- HTTP `Cache-Control` headers control TTL.
- Use for static assets and infrequently-changing API responses.
- Does not reduce server load until you have a large user base.

Layer 2: CDN (content delivery network)

- Sits between the client and your origin server, geographically distributed.
- Caches static assets (images, CSS, JS) close to the user.
- Can also cache API responses for public, non-personalized content.
- System Design Primer (Martin, 2017): push CDN vs. pull CDN. Pull is simpler to operate.

Layer 3: application cache (Redis / Memcached)

- Lives inside your infrastructure, in front of the database.
- Cache-aside pattern: application checks cache first, reads from DB on miss, writes result to cache.
- Redis: supports complex data structures (sorted sets, pub/sub), persistence, replication.
- Memcached: simpler, slightly faster for pure key-value workloads, no persistence.
- Hussein Nasser (2021): “Understanding how a database stores data on disk changes every decision you make about caching, indexing, and sharding.”

Cache invalidation: the hard part

- Write-through: write to cache and DB at the same time. Consistent. Slower writes.
- Write-back: write to cache, flush to DB asynchronously. Faster. Risk of data loss.
- Write-around: write directly to DB, bypass cache. Cache is populated on next read miss.
- Most interview answers want write-through for strong consistency, write-back for throughput.

Part 3: sharding

Why sharding exists

- A single database server has a ceiling: disk, CPU, network, lock contention.
- Sharding splits the dataset across multiple servers, each owning a subset.
- The sharding key determines which server owns which record.
- A bad sharding key creates hotspots: one server gets 90% of the traffic.

Range-based vs. hash-based sharding

```
Range-based:
  Shard 0: user IDs 0 – 999,999
  Shard 1: user IDs 1,000,000 – 1,999,999
  Problem: sequential inserts all land on one shard (hotspot)

Hash-based:
  Shard = hash(user_id) % num_shards
  Benefit: uniform distribution across shards
  Problem: range queries require scanning all shards
```

The hotspot tradeoff in interviews

- Hash-based sharding distributes writes evenly but kills range queries.
- Range-based sharding enables range scans but creates temporal hotspots.
- For URL shortener short codes: hash-based is correct (no range queries needed).
- For time-series data (logs, metrics): range-based on timestamp enables efficient time-range scans.
- State the tradeoff explicitly. The interviewer is listening for the reasoning, not the label.

Part 4: queues and async work

When synchronous calls break under load

- User uploads a photo. Synchronous path: resize → thumbnail → run ML tag → store → return 200.
- If any step takes 5 seconds, the HTTP request times out.
- Burst traffic: 10,000 uploads in 10 seconds overwhelms the resize service.
- The solution: write the upload to a queue immediately, return 200, process asynchronously.

What a message queue does

- Decouples the producer (app server) from the consumer (worker service).
- The queue absorbs bursts: the producer writes fast, the consumer processes at its own rate.
- Workers can be scaled independently of the API layer.
- If a worker crashes, the message stays in the queue until reprocessed.
- ByteByteGo 101 (2023): queues are the standard answer for any “what if processing is slow” question.

Queue options: Kafka vs. SQS

- **Kafka:** high-throughput, ordered within a partition, designed for streaming and replay. Use for event logs, analytics pipelines, real-time feeds.
- **SQS (AWS):** simpler, managed, at-least-once delivery, no replay. Use for task queues, background jobs, async processing in a single cloud environment.
- For most interview questions: state which one you'd choose and name one reason. The reason matters more than the brand.

Part 5: CAP theorem

The theorem in one sentence

- A distributed system can guarantee at most two of three properties: Consistency, Availability, and Partition tolerance.
- In practice, network partitions happen. You are choosing between consistency and availability.
- **CP systems:** return an error during a partition rather than a stale answer. Example: bank balance.
- **AP systems:** return a stale answer during a partition rather than an error. Example: social media feed.

How to use CAP in an interview

- Do not recite the theorem. Apply it.
- “For a URL shortener redirect, I can tolerate eventual consistency. A user who gets a slightly stale redirect still gets the right destination 99.9% of the time. I would choose an AP design.”
- “For a payment confirmation, I cannot tolerate stale reads. I need strong consistency. I would choose a CP design and accept that the service may return an error during a partition.”
- Martin (2017): consistency vs. availability is always a business decision, not a technical one.

> Understanding how a database actually stores data on disk changes every decision you make about caching, indexing, and sharding. The interview question is the easy part. Understanding why is the hard part.

<footer>Hussein Nasser, “System Designs” playlist, 2021</footer>

Take-aways

1. Choose SQL vs. NoSQL based on access pattern, not preference. Name the pattern first, then justify the choice.
2. Three caching layers serve different latencies and traffic types: client, CDN, and application cache (Redis/Memcached). Each has a cost.
3. Hash-based sharding distributes writes uniformly but disables range queries. Name the tradeoff when you choose a sharding key.
4. A message queue decouples producers from consumers and absorbs burst traffic. Reach for one whenever processing could be slow or bursty.
5. CAP is a business decision. State whether your use case tolerates stale reads (AP) or requires correctness under partition (CP).

What’s next

- **Reading:** Week 6 reading builds a URL shortener design end to end with worked math and a Mermaid component diagram
- **Section (this week):** System design lab: 45 minutes sketching, then peer critique on the senior-engineer rubric
- **HW 4 (out this week):** System design write-up for a well-known product, due week 8
- **Week 7:** Behavioral interviews and the full loop: STAR method, leadership principles, running an on-site